

macroexpand can be seen as a shorthand for writing the full (function (lambda...)) call.

pop

A more interesting function is the macro *pop*, which takes as its arguments a symbol whose value is a list. It reduces this list to its *cdr*, and returns the first element of the list as its value. In effect, *pop* treats the list like a stack, and pops off its top element. A call to *pop* is equivalent to executing the following piece of code:

```
> (prog1 (car stack) (setq stack (cdr stack)))
```

The function *prog1* evaluates all its arguments in sequence, returning the value of the first one. Thus, for any particular list, you can execute a line of code like the one shown above, which carries out the desired function. What you could do is define a macro that expands into code of this form. Thus you can get (*pop stack*) to expand into the *prog1* shown above.

pop is actually defined in Common Lisp. Here, we will write our own definition of the function, for the purpose of an example:

```
> (defmacro example-pop (stack)
  (list 'prog1
        (list 'car stack)
        (list 'setq stack (list 'cdr stack))))
EXAMPLE-POP
```

Let's see how the macro works:

```
> (setq stack '(a b c))
(A B C)

> (example-pop stack)
A

> stack
(B C)
```

example-pop returned the first element of *stack* as its value. In addition, it reduced *stack* to its *cdr*.

Note that it would be difficult to write this as an ordinary function, since it needs to get at the actual stack name to change its value. You would always have to quote the name of the stack you were passing, because arguments supplied to ordinary functions always get evaluated. However, with macros, this was not a problem.

Special functions and macros

Many of the so-called special functions are actually macros. Most notably, the functions *defun*, *prog*, *cond*, *setf* and *defmacro* itself are all Common Lisp macros. For example, a call to *setf* turns into a call to a more basic assignment function: a call to *setf*, whose first argument is a symbol, turns into a *setq*, as follows:

```
> (macroexpand '(setf a 'b))
(SETQ A 'B)
```

However, if the left-hand side of the call to *setf* is a call to the function *get*, the *setf* turns into a call to some internal Common Lisp property-list-changing function. Remember that the line between macros and special functions is rather thin. A Common Lisp implementation may implement some special functions as macros, and some macros as special forms—although it must still provide a macro definition in the latter case.

The use of macros is limited only by your own creativity. While you get creative with macros, I will tweak my example code-snippets for the next article, where I will discuss recursion, closures, and advanced macros. Enjoy playing God with the machine world. 

By: Vivek Shangari

The author is a hard-core hacker, if ever there was one. He always thinks programs, and holds a piece of code in his head all the time. You can reach him at vivek@codeCraft.in.

TechnoMail - Enterprise Email Server
Anti SPAM, Anti Virus, Email Content Filtering

Firewall, Internet Access Control
Content Filtering, Site Blocking

Bandwidth Management System

Managed Email Hosting Solutions

TechnoInfotech®

1, Vikas Permisses, 11 Bank Street, Fort, Mumbai, India - 400 001. Mobile: 09167399917. info@technoinfotech.com